

dotfiles

User Manual

Ghostty, tmux, zsh and Neovim on macOS, for running web apps
and for ROS 1 / ROS 2 work in Docker containers and on robots over SSH.

Repository	github.com/agus-darmawan/dotfiles
Author	Agus Darmawan
Platform	macOS 14+ (containers and robots: Ubuntu 20.04+)
Version	October 17, 2026

Contents

1 Overview **1**
- 1.1 The layers . . . 1
- 1.2 Where things run . . . 2
- 1.3 Conventions in this manual . . . 2
- 1.4 The repository . . . 3

2 Installation on macOS **4**
- 2.1 Before you start . . . 4
- 2.2 Install . . . 4
- 2.3 First launch . . . 5
- 2.4 Verify . . . 5
- 2.5 Update . . . 5
- 2.6 Uninstall . . . 6

3 Terminal applications **7**
- 3.1 Ghostty . . . 7
- 3.2 iTerm2 . . . 7
- 3.3 VS Code terminal . . . 7
- 3.4 Copy and paste . . . 8

4 The shell **9**
- 4.1 Prompt and title . . . 9
- 4.2 Line editing . . . 9
- 4.3 Navigation and files . . . 10
- 4.4 Git . . . 10
- 4.5 System helpers . . . 10

5 tmux **12**
- 5.1 Concepts . . . 12
- 5.2 The status bar . . . 12
- 5.3 Keys . . . 13
- 5.4 Sessions per project . . . 13
- 5.5 Nested tmux . . . 14

6 Neovim **15**
- 6.1 First steps . . . 15
- 6.2 Find and search . . . 15
- 6.3 Code . . . 16
- 6.4 Editing, buffers and windows . . . 16
- 6.5 Git . . . 17
- 6.6 Terminal and toggles . . . 17
- 6.7 Languages . . . 17

7 Web apps **19**
- 7.1 The app list . . . 19
- 7.2 The app command . . . 19

7.3 Cloudflare Tunnel . . . 20
7.4 PM2 and Docker . . . 22

8 ROS containers and robots **23**
8.1 The container . . . 23
8.2 Install the dotfiles inside . . . 23
8.3 SSH into the container . . . 24
8.4 ROS helpers . . . 24
8.5 ROS in Neovim . . . 25
8.6 Robots and Jetsons . . . 25

9 Customization **27**
9.1 Shell . . . 27
9.2 tmux . . . 27
9.3 Neovim . . . 28
9.4 SSH hosts and git identity . . . 28

10 Troubleshooting **30**
10.1 Terminal . . . 30
10.2 tmux . . . 30
10.3 Shell . . . 30
10.4 Web apps and tunnel . . . 31
10.5 Containers and Neovim . . . 31

A Quick reference **32**

CHAPTER 1

Overview

This repository is the complete terminal setup of a daily Mac: the terminal application, the multiplexer, the shell and the editor. All of them share one colour theme (Catppuccin Mocha), one font (JetBrainsMono Nerd Font) and know about each other.

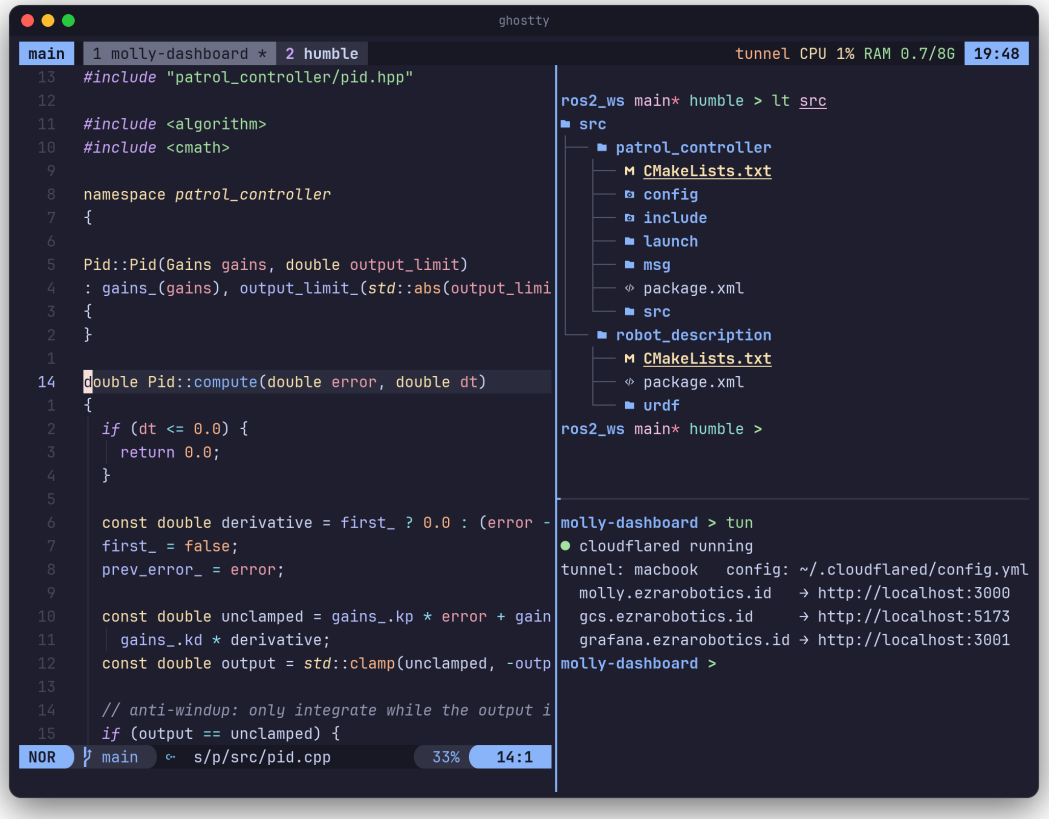


Figure 1.1: A tmux session on the Mac: the web app *molly-dashboard* runs in window 1, window 2 holds Neovim and a file tree inside the *humble* container (over SSH) and the tunnel status on the Mac.

1.1 The layers

Each tool does one job and they are stacked on top of each other:

Layer	What it does	How you reach it
Ghostty	The terminal application (the window). iTerm2 and the VS Code terminal work too.	Applications
tmux	Many terminals in one window: sessions, windows, split panes, popups, and the status bar with CPU, GPU, RAM, temperature and network speed.	`t`
zsh	The shell you type commands into, with completion, history search and the helpers described in this manual.	always running
starship	Draws the prompt: `dotfiles main* >`.	automatic
Neovim	The code editor, opened inside a tmux pane.	`nvim` or `v`

A typical start of the day: open Ghostty, type `t` to enter tmux, split the window with `C-a |`, run `nvim .` on one side and a dev server or `ssh humble` on the other.

1.2 Where things run

Machine	What runs there
Mac	Ghostty, tmux, zsh, Neovim, the web apps (PM2, Docker, `pnpm dev`), Cloudflare Tunnel. No ROS.
Dev containers	Ubuntu with ROS 2 Humble or Jazzy, or ROS 1 Noetic. zsh, Neovim and the ROS helpers. Reached with `ssh humble`, `ssh jazzy`, `ssh noetic`.
Robots, Jetsons	The same zsh and Neovim, installed with the Linux installer. Reached over SSH.

tmux always runs on the Mac. A pane that is connected over SSH shows the remote shell, while the status bar keeps showing the Mac. The tab title and the tmux window name show the host (`humble:~/ros2_ws`), so it is always clear where a pane is.

1.3 Conventions in this manual

- `C-a` means *Ctrl+a*, the tmux prefix. `C-a |` means: press `C-a`, release, then press `|`.
- `Space` in Neovim is the *leader* key. `Space ff` means: press Space, then `f`, then `f`. Press Space and wait to see every option.
- `M-` is the left Option key (Alt), `cmd` is the Command key.
- Commands in `this colour` are typed in the shell. Paths look like `~/.config/apps`.

1.4 The repository

Folder	Contents
`ghostty/`, `tmux/`, `zsh/`, `nvim/`, `starship/`, `git/`, `lazygit/`	Config packages, linked into ~ with GNU Stow
`iterm2/`, `vscode/`	iTerm2 profile and VS Code terminal settings
`ssh/`	Templates for `~/.ssh/config` and the host files
`templates/`	App list, ROS `.clangd` / `.clang-format`, colcon defaults
`install.sh`	macOS installer
`container/install.sh`	Linux installer for containers, Jetsons and robots
`tests/`	Smoke test and the ROS integration test used by CI
`docs/`	Markdown guides and this manual (`docs/manual/`)

CHAPTER 2

Installation on macOS

Apple Silicon or Intel, macOS 14 or newer. Most of the 10–20 minutes are Homebrew downloads.

2.1 Before you start

Install the Command Line Tools (git, clang, make). Homebrew asks for them too; installing them first avoids a second prompt:

```
xcode-select --install
```

Homebrew is installed by the installer when it is missing. Close running terminals and tmux if you already have configs for them: the installer moves the old files away.

2.2 Install

```
git clone git@github.com:agus-darmawan/dotfiles.git ~/dotfiles
cd ~/dotfiles && ./install.sh
```

Keep the repository at `~/dotfiles`; helpers and docs point there.

2.2.1 What the installer does

Step	Details
Homebrew bundle	Everything in the `Brewfile`: Ghostty, JetBrainsMono Nerd Font, tmux, starship, zoxide, fzf, eza, bat, fd, ripgrep, btop, macmon, Neovim and the tree-sitter CLI, node, python, git, delta, lazygit, gh, cloudflared, lazydocker. A failed package is reported and the install continues.
Backup	Moves existing `~/.config/{ghostty,tmux,nvim,zsh,lazygit,git}`, `~/.zshrc` and `~/.config/starship.toml` to `~/.dotfiles-backup/<date-time>/`.
Link	Symlinks every package with GNU Stow, so editing `~/.config/nvim/...` edits the repository.
tmux terminfo	Compiles `tmux-256color` into `~/.terminfo` (macOS ships without it).
iTerm2	Installs the profile *dotfiles (Catppuccin Mocha)*.
SSH	Creates `~/.ssh/config` (connection sharing) and the host files `~/.ssh/config.d/containers` and `~/.ssh/config.d/robots` when they do not exist. Never overwrites.
Apps	Creates `~/.config/apps`, the list of web apps (chapter 7).
Plugins	zsh plugins, and Neovim plugins at the versions pinned in `lazy-lock.json`.

Docker is not installed: use OrbStack (`brew install -cask orbstack`) or Docker Desktop.

2.2.2 Installer options

Command	Use it when
`./install.sh`	First install, or to pick up new Brewfile entries
`./install.sh -no-brew`	After `git pull`: re-link and refresh plugins, skip Homebrew
`./install.sh -link`	Only (re)create the symlinks
`./install.sh -unlink`	Remove the symlinks

Careful `container/install.sh` is for Linux only (containers, Jetsons, robots). Never run it on the Mac: it would put Linux binaries into `~/.local/bin`.

2.3 First launch

1. **Open Ghostty.** Font and theme are already set. If icons look like boxes, the font is missing: `brew install -cask font-jetbrains-mono-nerd-font`.
2. **Quick terminal.** System Settings → Privacy & Security → Accessibility → enable Ghostty. Then `cmd+‘` drops a terminal from the top of the screen in any app.
3. **Start tmux** with `t`. The status bar appears at the top.
4. **Start Neovim** with `nvim`. Mason installs the language servers and Treesitter compiles its parsers in the background, once. Watch with `:Mason` and `:Lazy`.
5. **Option key.** The left Option key works as Alt; the right one still types special characters.

2.4 Verify

```
~/dotfiles/tests/smoke.sh
```

It checks the links, the binaries, the tmux config and status segments, the zsh helpers, and loads every Neovim plugin. Every line should show a check mark. In Neovim, `:checkhealth` gives the detailed picture.

2.5 Update

Run the commands one at a time:

```
cd ~/dotfiles && git pull
./install.sh --no-brew        # re-link, plugins at the pinned versions
tmux kill-server              # then open a new tab and type: t
```

Component	Update with
Homebrew packages	`brewup` (or `brew bundle -file ~/dotfiles/Brewfile`)
Neovim plugins	`:Lazy sync`, then commit `lazy-lock.json`
LSP servers, tools	`:MasonUpdate`, then `:Mason` → `U`
Treesitter parsers	`:TSUpdate`

Component	Update with
zsh plugins	`plug-update`

2.6 Uninstall

```
cd ~/dotfiles && ./install.sh --unlink       # remove the symlinks
ls ~/.dotfiles-backup/                       # pick a backup
cp -R ~/.dotfiles-backup/<date-time>/. ~/    # restore the old configs
```

Plugin data lives in `~/.local/share/nvim`, `~/.local/state/nvim` and `~/.local/share/zsh`. Homebrew packages stay installed until you `brew uninstall` them.

CHAPTER 3

Terminal applications

The shell configuration is global, so every terminal application gets the same zsh, prompt and tools. Ghostty is the main one; iTerm2 and the VS Code terminal only need their font and title settings.

3.1 Ghostty

Config: `ghostty/.config/ghostty/config`. Reload it with `cmd+shift+,`.

Keys	Action
`cmd+‘`	Quick terminal: drops from the top of the screen, from any app (needs Accessibility permission)
`cmd+t` / `cmd+w`	New / close tab
`cmd+d` / `cmd+shift+d`	Native split right / down
`cmd+alt+h/j/k/l`	Move between native splits
`cmd+shift+enter`	Zoom the current split
`cmd+shift+e`	Equalize splits
`cmd+shift+,`	Reload the config

Native splits and tmux panes do the same job. This setup uses tmux, because tmux sessions keep running when the window closes and work the same over SSH.

3.2 iTerm2

`install.sh` copies the profile `iterm2/dotfiles.json` into iTerm2's DynamicProfiles folder. Open iTerm2 → Settings → Profiles, select **dotfiles (Catppuccin Mocha)** → **Other Actions...** → **Set as Default**. The profile brings the colours, the Nerd Font, left Option as Alt and lets the shell set the tab title. iTerm2 itself is not in the Brewfile: `brew install -cask iterm2`.

3.3 VS Code terminal

Open the command palette (`cmd+shift+p`) → **Preferences: Open User Settings (JSON)** and merge in `vscode/settings.json`. All settings start with `terminal.integrated.`:

Setting	Why
`fontFamily`	`JetBrainsMono Nerd Font Mono`, for `eza` icons
`tabs.title` = `${sequence}`	Tabs show the `host:path` title set by zsh
`macOptionIsMeta` = `true`	Option works as Alt (`M-b`, `M-f`, `M-c`)
`minimumContrastRatio` = `1`	Keeps the theme colours as they are

tmux is optional inside VS Code; plain zsh works fine there.

3.4 Copy and paste

Where	How
Ghostty, no tmux	Select with the mouse: it is copied automatically (`copy-on-select`). `cmd+c` also works. Paste with `cmd+v`.
Inside tmux	Drag with the mouse and release: the selection goes to the macOS clipboard. Hold `Option` while dragging to select freely across panes. Paste with `cmd+v`.
tmux, keyboard only	`C-a v` enters copy mode, move with `h/j/k/l`, `v` starts a selection, `C-v` toggles a block selection, `y` copies.
Neovim	`y` yanks to the macOS clipboard (`yy` a line, `v` then `y` a selection). `p` pastes in normal mode, `cmd+v` in insert mode.
Over SSH	The same. Neovim in a container or on a robot copies through OSC 52: container → SSH → tmux → Ghostty → macOS clipboard. Nothing to install.

CHAPTER 4

The shell

zsh is configured in `zsh/.config/zsh/`; the files load in name order (`00-env.zsh`, `10-options.zsh`, ..., `80-ros.zsh`). Machine-specific settings go into `~/.zshrc.local`, which is loaded last and not in git.

4.1 Prompt and title

The prompt is one line of plain ASCII, without icons:

```
dotfiles main* > _
patrol_controller main humble #42 12s > _
```

Part	Meaning
`dotfiles`	The current folder (last path component only)
`main*`	Git branch, then `*` modified, `+` staged, `?` untracked, `⇡`/`⇣` ahead/behind, `!` conflict
`humble #42`	ROS distro and `ROS_DOMAIN_ID`, or `noetic @go2` for a ROS 1 master. Only where ROS is sourced (containers, robots)
`12s`	How long the last command took, when it took more than 2 seconds
`>`	Green, or red after a failed command

The host is not in the prompt. Instead, zsh sets the window and tab title to `host:path`, for example `macbook:~/apps/molly` or `humble:~/ros2_ws`. Ghostty, iTerm2, VS Code and tmux show it, and a tmux window connected over SSH is named after the remote host.

4.2 Line editing

Keys	Action
`Tab`	Completion menu with fuzzy search and previews (folders, files, containers, SSH hosts); `<` / `>` switch groups
`Right` or `C-Space`	Accept the grey suggestion from history
`Up` / `Down`, `C-p` / `C-n`	Search history for what is already typed
`C-r`	Fuzzy search the whole history
`C-t`	Fuzzy-find a file and paste its path
`M-c`	Fuzzy `cd` into a subfolder
`C-x C-e`	Edit the current command in Neovim
`M-b` / `M-f`	Jump a word back / forward

4.3 Navigation and files

Command	Action
`cd <part>`	zoxide: jump to the best match among visited folders (`cd molly`)
`cdi`	zoxide, interactive
`..`	`cd ..` (typing a folder name alone changes into it)
`ls`, `ll`, `la`	eza with icons; long; long with hidden files and git status
`lt`, `lT`	Tree, 2 or 4 levels, git-ignored files hidden
`cat <file>`	bat, with syntax colours, no pager
`take <dir>`	`mkdir -p` and `cd`
`ex <archive>`	Extract tar, zip, gz, xz, zst or 7z
`v`, `vi`, `vim`	Neovim
`vc` / `dot`	Open the dotfiles in Neovim / `cd ~/dotfiles`

4.4 Git

Alias	Command
`g` / `lg`	git / lazygit
`gs`, `gd`, `gds`	Status, diff, staged diff
`ga`, `gaa`, `gc`, `gcm "msg"`, `gca`	Add, add all, commit, commit with message, amend without editing
`gco`, `gsw`, `gcb <name>`	Checkout, switch, new branch
`gl` / `gf`	Pull / fetch all and prune
`glog`	Graph of the last 20 commits
`gst` / `gstp`	Stash / stash pop

Diffs go through delta when it is installed and plain `less` otherwise (containers and robots without delta).

4.5 System helpers

Command	Action
`myip`	Local IP addresses (handy on robot networks)
`ports`	Listening TCP ports and their processes
`serve`	Static HTTP server in the current folder
`rsy <src> <dst>`	`rsync -avhP`
`top`	btop
`path`	One `$PATH` entry per line
`zr`	Reload zsh
`brewup`	Update all Homebrew packages
`flushdns`	Flush the macOS DNS cache
`plug-update`	Update the zsh plugins

Paste one command at a time When several lines are pasted at once and one of them is `exec zsh`, the shell is replaced and the lines after it are lost. Run commands that restart the shell on their own.

CHAPTER 5

tmux

tmux runs many terminals inside one window and keeps them alive when the window closes. Config: `tmux/.config/tmux/tmux.conf`; there are no plugins to install.

5.1 Concepts

Term	Meaning
Session	A named workspace, for example `main`, or one per project or per robot. Sessions keep running in the background.
Window	A tab inside a session, shown in the status bar as `1 molly`, `2 humble`.
Pane	A split inside a window.
Prefix	`C-a`: press it, release, then press the next key.

Start or re-attach with `t` (session `main`). Closing Ghostty does not stop tmux; type `t` again to get everything back.

5.2 The status bar

The bar sits at the top and is plain text:

```
main  1 molly  2 humble      tunnel CPU 12% GPU 5% RAM 9.1/24G 48C D 1.2M U 120K
   docker 2 BAT 85%+  14:22
```

Segment	Meaning
`main`	Session name. Blue normally, red while the prefix is pressed, grey when the outer tmux is turned off with `F12`
`1 molly`	Windows. `Z` zoomed pane, `*` activity in a background window, `S` panes typing in sync
`tunnel`	A `cloudflared` process is running (chapter 7)
`CPU 12%`	CPU load
`GPU 5%`	GPU busy, from `ioreg` (no extra tool)
`RAM 9.1/24G`	Memory used / total
`48C`	CPU temperature, from `macmon` (Apple Silicon). Appears a few seconds after tmux starts
`D 1.2M U 120K`	Download / upload speed per second on the default network interface
`docker 2`	Running containers (hidden when Docker is not running)
`BAT 85%+`	Battery; + means on power
`14:22`	Time

The bar refreshes every 3 seconds. Every segment is a function in `tmux/.config/tmux/scripts/status.sh`; run one by hand to debug it: `~/.config/tmux/scripts/status.sh temp`.

5.3 Keys

5.3.1 Panes and windows

Keys	Action
`C-a c`	New window in the current folder
`C-a |` or `C-a \`	Split right
`C-a -`	Split down
`C-h/j/k/l`	Move to the pane left/down/up/right; also moves between Neovim splits. In an SSH pane the key goes to the remote side
`C-a h/j/k/l`	Move pane, always the local tmux (also from SSH panes)
`C-a H/J/K/L`	Resize the pane (repeatable)
`C-a m`	Zoom the pane, again to restore
`C-a 1..9`	Go to window N
`C-a Tab`	Last window
`C-a n` / `C-a p`	Next / previous window
`C-a <` / `C-a >`	Move the window left / right
`C-a ,`	Rename the window
`C-a x` / `C-a X` / `C-a Q`	Close pane / window / session (asks first)
`C-a y`	Toggle typing into all panes of the window
`C-a v`	Copy mode (section 3.4)
`C-a C-l`	Clear the screen
`C-a r`	Reload the config

5.3.2 Popups and pickers

Keys	Action
`C-a f`	Sessionizer: pick a project folder, open or switch to a session named after it
`C-a s`	Sessions and windows as a tree; `x` closes the selected one
`C-a S`	SSH hosts: `enter` new window, `ctrl-v` / `ctrl-s` split, `ctrl-o` here
`C-a d`	Running Docker containers: open a shell inside
`C-a a`	Web apps: start one in a new window (chapter 7)
`C-a g`	lazygit in the current folder
`C-a D`	lazydocker
`C-a b`	btop
`C-a t`	Floating scratch terminal; its content stays when it is closed
`F12`	Turn the outer tmux off and on (for a tmux running on a robot)

5.4 Sessions per project

The sessionizer (`C-a f`, or `ts` in the shell) lists folders under `~/apps`, `~/web`, `~/Projects`, `~/projects`, `~/dev`, `~/workspace`, `~/Developer`, `~/ros2_ws`, `~/dotfiles` and `~/Documents/GitHub`. Picking one opens a session named after the folder, or switches to it when it exists.

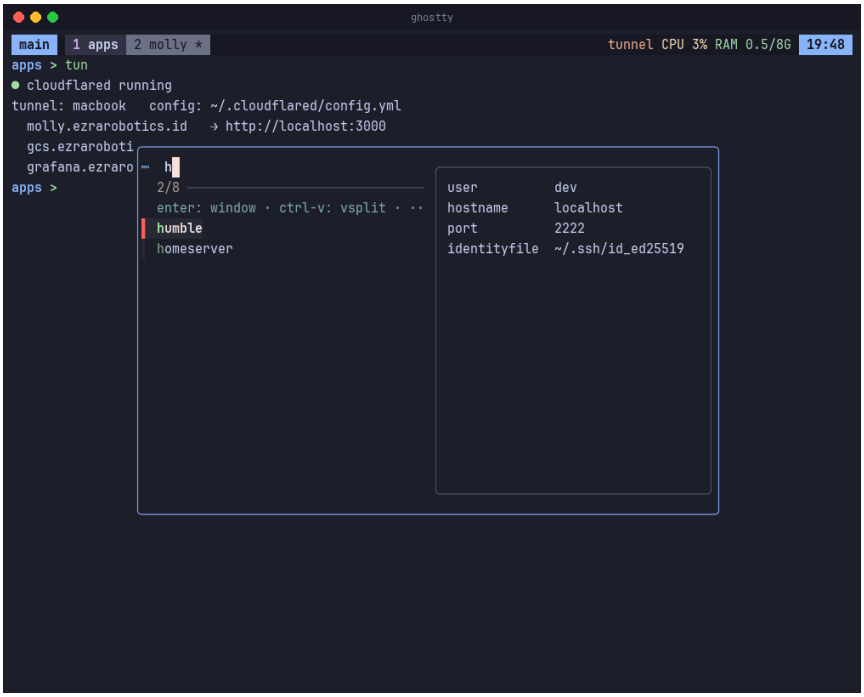


Figure 5.1: `C-a S`: SSH host picker with the connection details of the selected host.

Command	Action
`t`	Attach or create the session `main`
`ta`, `tl`, `tk <name>`	Attach, list, kill a session
`ts`	Sessionizer
`dev <host>`	Session named after an SSH host with windows `code`, `build`, `run`, each already connected (chapter 8)

5.5 Nested tmux

When you SSH into a robot that runs its own tmux, the keys would go to the Mac's tmux. Press `F12`: the session name turns grey, the bar shows `OFF(F12)` and every key goes to the robot's tmux. Press `F12` again to come back. The robot's tmux keeps its default prefix `C-b`, so the two never clash.

CHAPTER 6

Neovim

Neovim is configured in `nvim/.config/nvim/` with about twenty plugins managed by lazy.nvim: blink.cmp (completion), snacks.nvim (pickers, explorer, terminal, lazygit), which-key, gitsigns, conform (formatting), nvim-lspconfig with Mason, nvim-treesitter, flash, mini.surround and mini.pairs.

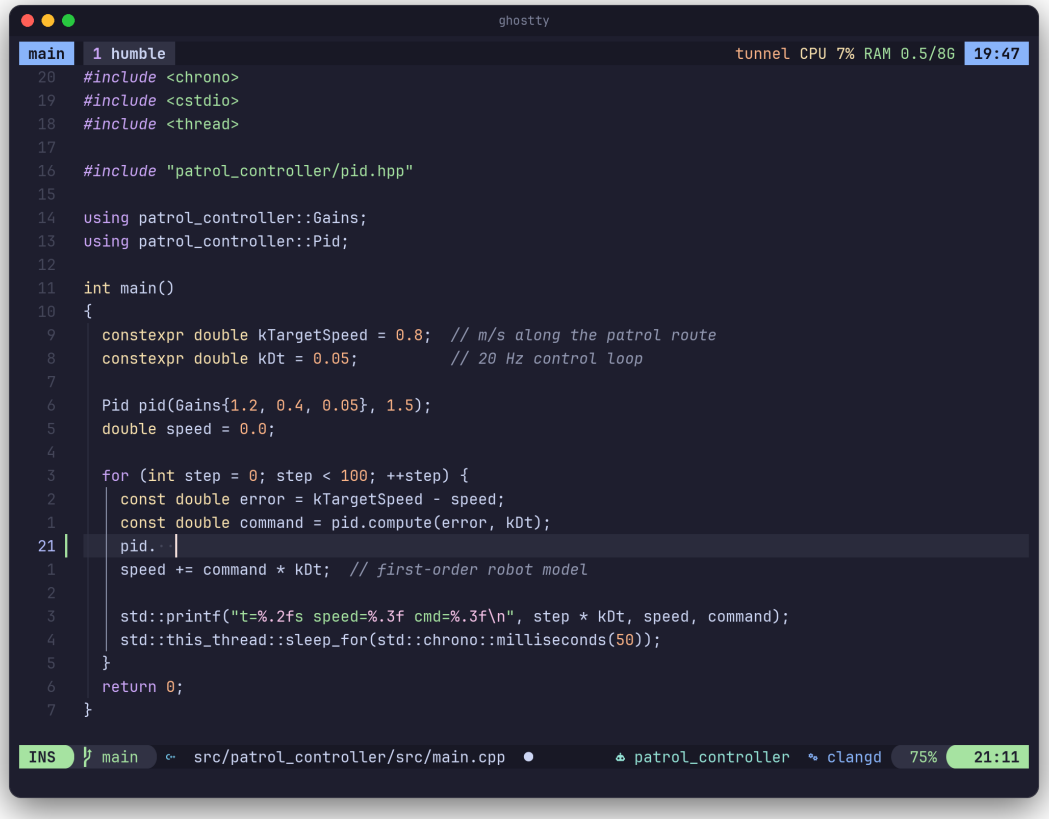


Figure 6.1: clangd completion with the signature of the selected member, in a container over SSH.

6.1 First steps

- New to Vim motions? Run `:Tutor` first (about 30 minutes).
- Press `Space` and wait: which-key lists every group and key. `Space sk` searches all keymaps.
- `:Lazy` shows the plugins, `:Mason` the language servers and tools, `:checkhealth` what is missing.

6.2 Find and search

Keys	Action
`Space Space`	Smart file finder (recent and frequent files first)
`Space /`	Grep the project
`Space ,`	Open buffers
`Space e`	File explorer on the right (`a` add, `d` delete, `r` rename)
`Space ff`, `fg`, `fr`, `fp`, `fc`	Files, git files, recent, projects, Neovim config
`Space fy`	Copy the file path
`Space sw`	Grep the word under the cursor or the selection
`Space sb`, `sh`, `sk`, `sc`	Buffer lines, help, keymaps, commands
`Space sd`, `sD`	Diagnostics, of the project or the buffer
`Space su`, `sr`, `sq`	Undo history, resume the last picker, quickfix list
`Space :`	Command history

6.3 Code

These keys work in a buffer with a language server attached.

Keys	Action
`gd`, `gD`, `gr`, `gI`, `gy`	Definition, declaration, references, implementation, type definition
`K` / `gK`	Hover documentation / signature help (`C-k` in insert mode)
`Space ca`	Code action (imports, quick fixes)
`Space cr`	Rename the symbol in the whole project
`Space cR`	Rename the file and update imports
`Space cf`	Format (format on save is on; `Space uf` toggles it)
`Space cd`	Diagnostics of the current line
`Space ch`	Switch between header and source (C/C++)
`Space ss` / `Space sS`	Symbols in the buffer / workspace
`]d` `[d`, `]e` `[e`	Next / previous diagnostic, error
`]]` `[[`	Next / previous reference of the word under the cursor
`Space xx` / `Space xq`	Project diagnostics / toggle the quickfix list
`Space cl`	LSP health check
`Space cm`	Mason

Completion: `Tab` / `S-Tab` select, `Enter` accepts, `C-Space` opens the menu.

6.4 Editing, buffers and windows

Keys	Action
`s` + two letters	Flash: jump anywhere on the screen
`S`	Select a Treesitter node (function, block)
`gsa`, `gsd`, `gsr`	Surround add, delete, replace (`gsaiw"` wraps a word in quotes)
`gcc` / `gc`	Comment a line / a selection
`A-j` / `A-k`	Move lines down / up

Keys	Action
`C-s`	Save
`S-h` / `S-l`	Previous / next buffer
`Space bd`, `bD`, `bb`	Delete buffer, delete other buffers, alternate buffer
`Space -` / `Space |`	Split below / right
`C-h/j/k/l`	Move between splits, and on into tmux panes
`Space wd` / `Space wo`	Close the window / keep only this window
`Space qq`	Quit everything

6.5 Git

Keys	Action
`Space gg`	lazygit (`space` stage, `c` commit, `P` push, `?` help)
`Space gl` / `Space gf`	lazygit log / history of this file
`Space gs` / `Space gb`	Git status / branches
`]h` / `[h`	Next / previous changed hunk
`Space ghs`, `ghr`, `ghp`, `ghb`	Stage, reset, preview, blame a hunk
`Space ub`	Inline blame on / off
`Space go`	Open the line on GitHub

6.6 Terminal and toggles

Keys	Action
`C-/`	Toggle a terminal at the bottom
`Space tt` / `Space tv`	Floating / side terminal
`Space td` / `Space tb`	lazydocker / btop
`Space uf`	Format on save on / off
`Space uh`, `ud`, `uw`, `us`	Inlay hints, diagnostics, wrap, spelling
`Space ul`, `uL`, `ug`, `uD`	Line numbers, relative numbers, indent guides, dim
`Space un`	Dismiss notifications

6.7 Languages

Every language is registered in one file, `lua/config/languages.lua`: the Treesitter parser, the language server and the formatter.

Language	Server, formatter
C / C++	clangd; clang-format, only when the project has a `.clang-format`
Python	basedpyright, ruff (lint and format)
CMake	neocmake

Language	Server, formatter
ROS XML, URDF, xacro, launch	lemminx
`.msg` / `.srv` / `.action`	Syntax highlighting
TypeScript, React, Next.js	vtsls, ESLint, Tailwind CSS; biome when `biome.json` exists, otherwise prettier
HTML, CSS, JSON, YAML, TOML	html, cssls, jsonls (with SchemaStore), yamlls, taplo
Bash / zsh	bashls, shfmt
Docker	dockerls
Dart	dart language server when the Dart SDK is installed
Lua	lua_ls, stylua

On machines without internet (robots), `export NVIM_OFFLINE=1` in `~/.zshrc.local` skips every automatic download of parsers, servers and plugin updates.

CHAPTER 7

Web apps

Three helpers run the web apps on the Mac or a home server, defined in `zsh/.config/zsh/50-web.zsh`:

Helper	For
`app`	Start an app in its own tmux window, from a short list of your projects
`tun`	See and change what the Cloudflare Tunnel serves, or share a port for a moment
`pml pmlog pmr pms`	PM2 shortcuts

7.1 The app list

`install.sh` creates `~/.config/apps`. One line per app: a name, its folder, and optionally the command that starts it.

```
# name        folder                    [command]
molly         ~/apps/molly-dashboard
gcs           ~/apps/nextgen-gcs
netra         ~/apps/netra              scripts/local.sh up
database      ~/apps/database
```

Without a command, the folder decides how the app starts:

The folder has	app runs
`ecosystem.config.js` (or `.cjs`, `.json`)	`pm2 start <file>`, then follows `pm2 logs`
`compose.yaml` or `docker-compose.yml`	`docker compose up -d`, then follows the logs
`package.json`	`pnpm dev`, `bun run dev`, `yarn dev` or `npm run dev`, picked from the lockfile
none of these	Only opens a shell in the folder

7.2 The app command

Command	Action
`app` or `C-a a`	Pick an app: a new tmux window named after it opens in its folder and starts it
`app <name>`	The same without the picker
`app ls`	Every app with how it starts (`pm2`, `compose`, `node`, `cmd`) and its folder
`app up <name>`	Start in the background (PM2 and compose)
`app down <name>`	`pm2 stop` or `docker compose down`

Command	Action
`app logs <name>`	PM2 or compose logs
`app cd <name>`	Change into the folder

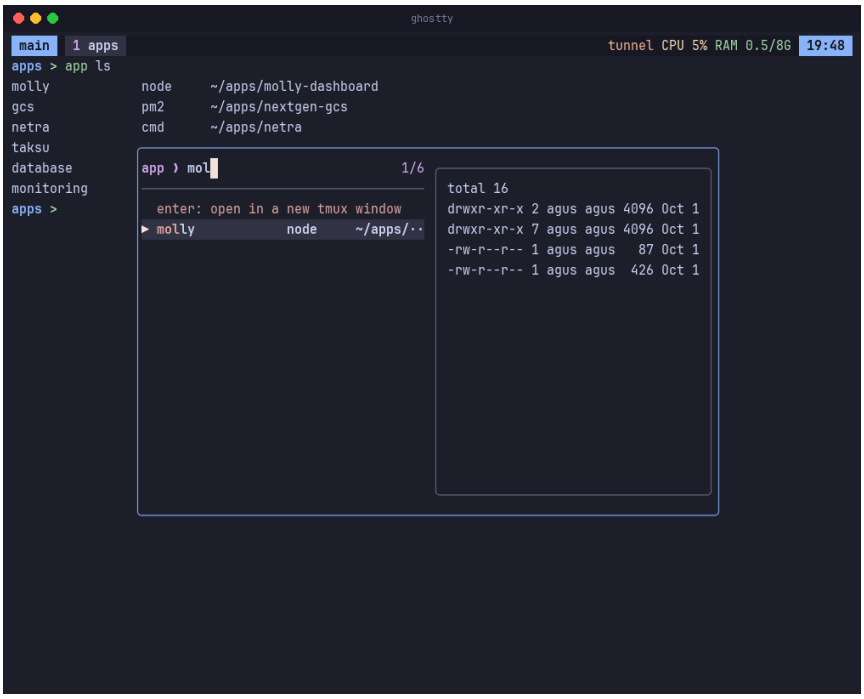


Figure 7.1: `C-a a`: the app picker, with the folder of the selected app on the right.

A typical morning:

```
t                 # tmux session main
app molly         # window "molly": pnpm dev
app database      # window "database": docker compose up -d, then the logs
tun               # are the public hostnames up?
```

7.3 Cloudflare Tunnel

`tun` works on a locally managed tunnel: one `config.yml` with ingress rules, run by `cloudflared` as a service. Tunnels managed in the Cloudflare dashboard (token based) have no local config; change those in the dashboard.

7.3.1 One-time setup

```
cloudflared tunnel login          # browser login, writes ~/.cloudflared/cert.pem
cloudflared tunnel create mac     # writes ~/.cloudflared/<tunnel-id>.json
```

Then write `~/.cloudflared/config.yml`:

```
tunnel: <tunnel-id>
credentials-file: /Users/<you>/.cloudflared/<tunnel-id>.json

ingress:
  - hostname: molly.example.com
```

```
    service: http://localhost:3000
  - service: http_status:404      # must stay last: tun adds routes above it
```

```
cloudflared tunnel route dns mac molly.example.com
brew services start cloudflared   # starts at login, reads ~/.cloudflared/config.yml
```

Careful Start the service as your user, not with sudo: a root service reads its config from `/etc/cloudflared`, not from your home folder.

7.3.2 Commands

Command	Action
`tun`	Running or not, the tunnel name, and every hostname → service route
`tun add <host> <port>`	Add a route (a bare port becomes `http://localhost:<port>`), create its DNS record, restart
`tun rm <host>`	Remove a route and restart (delete the DNS record in the dashboard)
`tun test <host>`	Which rule a request for that hostname would hit
`tun share <port>`	Temporary `*.trycloudflare.com` URL for a local port; `C-c` ends it
`tun log`	Follow the cloudflared log
`tun restart`	Restart the service
`tun edit`	Open `config.yml` in Neovim



Figure 7.2: `tun`: tunnel status and the hostname routes.

Every change copies the config to `config.yml.bak` first and runs `cloudflared tunnel ingress validate` afterwards; an invalid result is rolled back.

Settings, in `~/.zshrc.local`:

Variable	Default
CLOUDFLARED_CONFIG	~/.cloudflared/config.yml
CLOUDFLARED_RESTART	brew services restart cloudflared

With cloudflared service install instead of Homebrew:

```
export CLOUDFLARED_RESTART='launchctl kickstart -k gui/$(id -u)/com.cloudflare.
    cloudflared'
```

7.4 PM2 and Docker

Alias	Command
pml	pm2 ls
pmlog	pm2 logs -lines 100
pmr <name>	pm2 restart <name>
pms	pm2 save (the list that pm2 resurrect restores)
dps, dpsa, di	Containers, all containers, images
dcu, dcd, dcl, dcb	docker compose up -d, down, logs -f, build
dsh [container]	Shell into a container (zsh, bash or sh), fuzzy pick without a name
dlog, dstat	Follow logs, live CPU/RAM per container
dstop, drm	Stop / remove containers (multi-select with Tab)
dclean	Prune stopped containers, networks, dangling images and volumes
lzd or C-a D	lazydocker

To start the saved PM2 apps at login: pm2 startup once, then pm2 save whenever the list changes.

CHAPTER 8

ROS containers and robots

ROS lives in Ubuntu containers (ROS 1 Noetic on 20.04, ROS 2 Humble on 22.04, Jazzy on 24.04) and on the robots. The Mac has no ROS at all: tmux runs on the Mac and its panes SSH into a container or a robot, where zsh, Neovim and the ROS helpers are installed.

8.1 The container

Share the configuration (mount the repository), never the data: plugins and Mason binaries are built per operating system, so they live in volumes.

```
# compose.yaml in the ROS workspace
services:
  humble:
    build: .
    container_name: humble
    network_mode: host
    working_dir: /home/dev/ros2_ws
    environment:
      - ROS_DOMAIN_ID=${ROS_DOMAIN_ID:-0}
    volumes:
      - ./:/home/dev/ros2_ws
      - ~/dotfiles:/home/dev/dotfiles          # config, shared with the Mac
      - nvim-data:/home/dev/.local/share/nvim  # plugins + Mason (Linux binaries)
      - zsh-data:/home/dev/.local/share/zsh    # zsh plugins + history
volumes:
  nvim-data:
  zsh-data:
```

The image needs a `dev` user and an SSH server, one port per container: 2222 humble, 2223 jazzy, 2224 noetic. With bridge networking, keep port 22 and publish it on localhost only, with `ports: ["127.0.0.1:2222:22"]` in the compose file.

```
FROM ros:humble                          # or ros:jazzy, ros:noetic
RUN apt-get update && apt-get install -y openssh-server sudo && \
    useradd -m -s /bin/bash -G sudo dev && \
    echo "dev ALL=(ALL) NOPASSWD:ALL" >/etc/sudoers.d/dev && \
    mkdir -p /run/sshd && sed -i 's/^#\?Port .*/Port 2222/' /etc/ssh/sshd_config
COPY --chown=dev:dev authorized_keys /home/dev/.ssh/authorized_keys
CMD ["/usr/sbin/sshd", "-D"]
```

Host networking on a Mac works out of the box in OrbStack; Docker Desktop needs it switched on in its settings.

8.2 Install the dotfiles inside

Once per container; the volumes keep the result across rebuilds:

```
docker exec -it -u dev humble bash
~/dotfiles/container/install.sh       # 2-4 minutes, x86_64 or arm64
```

It installs zsh, Neovim, fzf, starship, zoxide, lazygit, Node and eza, links the `zsh`, `nvim`, `starship`, `git` and `lazygit` configs, installs the plugins and makes zsh the login shell. Options: `-no-apt` when the image already has the apt packages, `-minimal` to skip Node (faster, but language servers that need npm will not install).

On Ubuntu 20.04 (glibc 2.31) the installer takes Neovim from `neovim/neovim-releases`, built for old glibc, and skips the tree-sitter CLI; Neovim then uses its bundled parsers.

8.3 SSH into the container

`install.sh` on the Mac created `~/.ssh/config.d/containers`:

Host	Container
`humble`	ROS 2 Humble, Ubuntu 22.04, `localhost:2222`
`jazzy`	ROS 2 Jazzy, Ubuntu 24.04, `localhost:2223`
`noetic`	ROS 1 Noetic, Ubuntu 20.04, `localhost:2224`

All of them connect as `dev`, forward your SSH agent (so `git push` works inside) and do not pin host keys, because containers are rebuilt often. Thanks to connection sharing, the first connection authenticates and every further pane to the same host opens instantly.

Keys / command	Action
`ssh humble`	One shell in the container
`dev humble`	tmux session `humble` with windows `code`, `build`, `run`, each already in the container
`C-a S` or `s`	Pick any host
`C-a d` or `dsh`	`docker exec` shell in a running container, no SSH needed

8.4 ROS helpers

The helpers exist only where `/opt/ros` exists. The installed distro is sourced automatically (`export ROS_AUTO=0` turns that off) and the helpers know whether they run under ROS 1 or ROS 2.

Command	Action
`ros [distro]`	Source `/opt/ros/<distro>` and the nearest workspace overlay
`rosws`	Print the workspace root, also before the first build
`cb` / `cbp <pkg>`	Build the workspace / packages with their dependencies; writes `compile_commands.json` for clangd. colcon on ROS 2; `catkin build` or `catkin_make` on ROS 1
`ct [pkg]`	Run the tests and print the results
`rcl`	Remove `build/install/devel/log/` (asks first)
`rosdomain <id>`	ROS 2: set `ROS_DOMAIN_ID`, shown in the prompt as `#id`
`rosmaster <host>`	ROS 1: set `ROS_MASTER_URI` and `ROS_IP`, shown as `@host`

Command	Action
`rtopic` / `rnode`	Pick a topic and echo it / pick a node and show its info
`rtl`, `rte`, `rnl`	Topic list, topic echo, node list
`rl`, `rr`, `rbag`	Launch, run, bag

8.5 ROS in Neovim

The `Space r` menu appears only where ROS is available (a sourced distro, or colcon or catkin on the `PATH`). Start Neovim from a shell where ROS is sourced.



Figure 8.1: `Space r`: the ROS menu in which-key, inside the *humble* container.

Keys	Action
`Space rb`	Build the package of the current file in the background: errors go to the quickfix list, `compile_commands.json` is merged and clangd restarts
`Space rB`	Build the whole workspace
`Space rL`	Last build log
`Space rc`	Merge `compile_commands.json`
`Space rt`	Topics: echo, hz or info
`Space rn`	Nodes: info
`Space rl`	Pick a launch file: `ros2 launch` / `roslaunch`
`Space rr`	Pick an executable: `ros2 run` / `rosrun`

Commands: `:RosBuild [pkg]`, `:RosLog`, `:RosCompileDb`.

For clangd, copy `.clangd` and `.clang-format` from `templates/ros2/` into the workspace root. The container installer also writes `~/.colcon/defaults.yaml`, so a plain `colcon build` exports compile commands too.

8.6 Robots and Jetsons

The same installer works directly on a robot:



Figure 8.2: `Space rb` with a compiler error: the error is in the quickfix list and shown on the line; `]q` / `[q` jump between errors.

```
git clone https://github.com/agus-darmawan/dotfiles ~/dotfiles
~/dotfiles/container/install.sh --minimal
```

On the Mac, add the robot to `~/.ssh/config.d/robots`, copy your key once with `skey go2`, then:

Command	Action
`dev go2`	tmux session on the robot (windows code, build, run)
`sall go2 lite3`	One window, one pane per robot, typing goes to all of them (`C-a y` toggles)
`sfwd <host> <port>`	Forward a remote port (dashboard, Foxglove) to `localhost`
`skey <host>`	Copy your ed25519 key to a host

Without internet on the robot, add `export NVIM_OFFLINE=1` to its `~/.zshrc.local`. When the robot runs its own tmux, press `F12` on the Mac (section 5).

CHAPTER 9

Customization

Two ways to change things:

- **Edit the repository** for changes on every machine. Configs are symlinked, so editing `~/.config/nvim/...` edits `~/dotfiles/...`; commit and push as usual.
- **Local files** for one machine only. They are not in git:

File	Read by
`~/.zshrc.local`	zsh, last
`~/.config/tmux/tmux.local.conf`	tmux, after the main config
`~/.config/git/config.local`	git, included at the end
`~/.config/nvim/lua/config/local.lua`	Neovim, after everything else
`~/.ssh/config.d/*`	ssh
`~/.config/apps`	app

9.1 Shell

```
# ~/.zshrc.local
# folders searched by the sessionizer (C-a f)
export TMUX_SESSIONIZER_PATHS="$HOME/apps $HOME/web $HOME/ros2_ws"
# another tunnel config for tun
export CLOUDFLARED_CONFIG=~/.cloudflared/home.yml
# containers: don't source ROS automatically
export ROS_AUTO=0
# robots without internet: no automatic downloads in Neovim
export NVIM_OFFLINE=1
```

- Shared aliases go into `zsh/.config/zsh/40-aliases.zsh`.
- More zsh plugins: add `plug <github-user>/<repo>` to `20-plugins.zsh`.
- The prompt is `starship/.config/starship.toml`; add `$hostname` to its `format` to show the host again.

9.2 tmux

```
# ~/.config/tmux/tmux.local.conf
set -g prefix C-b                # back to the default prefix
bind C-b send-prefix
set -g status-position bottom    # status bar at the bottom
```

Status bar segments. `status-right` in `tmux.conf` calls `scripts/status.sh` once per segment: `tunnel`, `cpu`, `gpu`, `mem`, `temp`, `netspeed`, `docker`, `battery` (and `net`, the local IP, not shown by default). To add one, write a function and a line for it in the `case` at the bottom of `status.sh`:

```
pm2_seg() {   # online PM2 processes
  command -v pm2 >/dev/null || return 0
  printf 'pm2 %s ' "$(pm2 jlist | jq '[.[] | select(.pm2_env.status == "online")] |
    length')"
}
case "$1" in
  pm2) pm2_seg ;;
  # ...
```

Then add `#($S/status.sh pm2)` to `status-right` and reload with `C-a r`.

9.3 Neovim

Add a language in `lua/config/languages.lua`: the parser in `M.treesitter`, the server in `M.servers()`, the formatter in `M.formatters_by_ft`. Restart Neovim; Mason installs the server.

Add a plugin with a new file in `lua/plugins/` that returns a lazy.nvim spec; `:Lazy` installs it and `lazy-lock.json` records the version.

What	Where
General keymaps	`lua/config/keymaps.lua`
Keymaps of a plugin	The `keys` of its spec in `lua/plugins/`
LSP keymaps	`lua/plugins/lsp.lua`
ROS keymaps and commands	`lua/util/ros.lua`
Options	`lua/config/options.lua`
Theme	`lua/plugins/colorscheme.lua` (latte, frappe, macchiato, mocha)

```
-- ~/.config/nvim/lua/config/local.lua  (this machine only)
vim.g.autoformat = false
vim.opt.relativenumber = false
```

9.4 SSH hosts and git identity

Any file in `~/.ssh/config.d/` is read, so hosts can be split by project; every host appears in `s` and `C-a S`.

```
# ~/.ssh/config.d/robots
Host go2
  HostName 192.168.123.18
  User unitree

Host go2-jetson
  HostName 192.168.123.15
  User unitree
  ProxyJump go2
```

A different git identity for work repositories:

```
# ~/.config/git/config.local
[includeIf "gitdir:~/ezra/"]
    path = ~/.config/git/work
```

CHAPTER 10

Troubleshooting

Run `~/dotfiles/tests/smoke.sh` first: it usually points at the broken part.

10.1 Terminal

Problem	Fix
Icons show as boxes or ?	The Nerd Font is missing or not selected: `brew install -cask font-jetbrains-mono-nerd-font`, restart the terminal
`cmd+‘` does nothing	Give Ghostty Accessibility permission and restart it
`M-c`, `M-b` type special characters	Use the left Option key. iTerm2: select the *dotfiles* profile. VS Code: `macOptionIsMeta`
Tab title shows `zsh`, not `host:path`	iTerm2: the *dotfiles* profile. VS Code: `tabs.title = ${sequence}`
Colours or backspace wrong over SSH	Ghostty installs its terminfo on the remote host; otherwise `export TERM=xterm-256color` there

10.2 tmux

Problem	Fix
No status bar	You are not in tmux yet: type `t`
Old bar with icons after an update	The old tmux server still runs: `tmux kill-server`, then `t`
`missing or unsuitable terminal: tmux-256color`	`./install.sh -no-brew` compiles the terminfo
No temperature	`brew install macmon` (Apple Silicon); the value appears a few seconds after tmux starts
No `D/U`	Needs two refreshes (about 6 seconds); follows the default route’s interface
A segment is empty	Run it by hand: `~/.config/tmux/scripts/status.sh gpu`
`C-h/j/k/l` in an SSH pane stays inside	On purpose: the key goes to the remote side. `C-a h/j/k/l` always moves the local pane
Keys go to the wrong tmux on a robot	`F12` turns the outer tmux off and on
An SSH window keeps the Mac’s name	The remote shell does not set a title; rename it with `C-a ,`

10.3 Shell

Problem	Fix
Pasted commands stop after `exec zsh`	`exec zsh` replaces the shell; run it on its own
`zsh compinit: insecure directories`	`compaudit | xargs chmod g-w,o-w`
Plugins not loading	Delete the broken one in `~/.local/share/zsh/plugins` and open a new shell
`cd` behaves oddly	`cd` is zoxide; `builtin cd <path>` changes literally
`ros`, `cb` not found on the Mac	On purpose: ROS helpers exist only where `/opt/ros` exists
`nvim`: `exec format error`	`container/install.sh` was run on the Mac. Remove the Linux binaries: `file -L ~/.local/bin/* | grep ELF`

10.4 Web apps and tunnel

Problem	Fix
`app: 'x' is not in ~/.config/apps`	Add a line `x ~/path/to/x [command]`; `app ls` shows the list
`app` opens a window but nothing runs	The folder has no `package.json`, compose or PM2 file: add the start command
`tun add`: no catch-all rule	The config needs `- service: http_status:404` as the last rule
Hostname does not resolve	The DNS step needs `~/.cloudflared/cert.pem` (`cloudflared tunnel login`)
`tun restart` does not fit your setup	Set `CLOUDFLARED_RESTART` in `~/.zshrc.local`

10.5 Containers and Neovim

Problem	Fix
`ssh humble` refused	The container must run sshd on `127.0.0.1:2222`
`GLIBC_2.34 not found`	Re-run `container/install.sh`: it picks the build for old glibc
`git diff`: `delta: not found`	Pull the latest dotfiles; git falls back to `less`
No completion, `gd` does nothing	`:checkhealth vim.lsp`; is the server in `:Mason`?
clangd does not know ROS headers	Build once (`cbp <pkg>` or `Space rb`), then `Space rc`
`Space r` keys missing	Only where ROS is available; start Neovim from a sourced shell
Treesitter parsers do not install	Needs the tree-sitter CLI and a C compiler (`xcode-select -install`), then `:TSUpdate`
Something broke after `:Lazy sync`	`:Lazy restore` returns to `lazy-lock.json`

APPENDIX A

Quick reference

tmux (prefix `C-a`)

Keys	Action
`t`	Attach or create session `main`
`C-a |` / `C-a -`	Split right / down
`C-h/j/k/l`	Move between panes and Neovim splits
`C-a c`, `C-a 1..9`	New window, go to window
`C-a m`	Zoom pane
`C-a f` / `C-a s`	Open a project / switch session
`C-a S` / `C-a d`	SSH to a host / shell into a container
`C-a a`	Start a web app
`C-a g`, `C-a D`, `C-a b`	lazygit, lazydocker, btop
`C-a v`	Copy mode
`F12`	Outer tmux off / on

Neovim (leader `Space`)

Keys	Action
`Space Space` / `Space /`	Find files / grep
`Space e`	Explorer
`gd`, `gr`, `K`	Definition, references, hover
`Space ca` / `Space cr`	Code action / rename
`Space rb`	Build this ROS package (containers)
`Space gg`	lazygit
`Space sk`	Search keymaps

Shell

Command	Action
`cd <part>`, `C-r`, `C-t`	Jump to a folder, search history, find a file
`app <name>`, `app ls`	Start a web app, list apps
`tun`, `tun add <host> <port>`	Tunnel status, add a hostname
`ssh humble`, `dev humble`	Shell / tmux session in a container
`cb`, `cbp <pkg>`	Build the ROS workspace / a package

Command	Action
dps, dsh, lzd	Containers, shell into one, lazydocker
gs, gd, lg	git status, diff, lazygit
myip, ports	Local IPs, listening ports

Files

File	Purpose
~/.config/apps	Web apps for app
~/.cloudflared/config.yml	Tunnel routes for tun
~/.ssh/config.d/containers	Container hosts
~/.ssh/config.d/robots	Robot hosts
~/.zshrc.local	Shell settings for this machine
~/.dotfiles-backup/	Configs replaced by the installer